

Python

# MongoDB и Python. Модули



# Преподаватель

Портрет

**Имя Фамилия**

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

# Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

# Повторение

-  Работа с MySQL из Python
-  Подключение к базе данных
-  Работа с курсором
-  Получение результатов запроса
-  Параметризованные запросы
-  Контекстный менеджер with
-  Метод commit

# План занятия

- Подключение к базе MongoDB из Python
- Добавление данных
- Чтение данных
- Обновление данных
- Удаление данных
- Обработка ошибок
- Основные типы исключений
- Модули



# ОСНОВНОЙ БЛОК





# Подключение к базе MongoDB из Python



## MongoDB

Это документоориентированная база данных, с которой можно удобно работать и из Python



# Библиотека pymongo



## Пояснения

Для работы с MongoDB в Python используется библиотека pymongo, которая позволяет выполнять запросы к базе в привычной форме: через словари, как в консоли MongoDB

## Установка

```
pip install pymongo
```

# Подключение к серверу



Для работы с базой данных MongoDB необходимо создать **объект** MongoClient, передав в него строку подключения. Она содержит все параметры для подключения: логин, пароль, адрес сервера и настройки авторизации.

# Подключение к серверу

```
from pymongo import MongoClient

client = MongoClient(
    "mongodb://ich_editor:verystrongpassword"
    "@mongo.itcareerhub.de/?readPreference=primary"
    "&ssl=false&authMechanism=DEFAULT&authSource=ich_edit"
)
```

## Основные параметры:

- `ich_editor` — имя пользователя
- `verystrongpassword` — пароль
- `mongo.itcareerhub.de` — адрес сервера
- `authSource=ich_edit` — имя базы, в которой хранятся учётные данные

# Проверка подключения

```
client.admin.command("ping")  
print("Connection successful!")
```

# Выбор базы данных



## Пример

```
db = client["ich_edit"]
```

## Особенности

- Если база не существует, она будет создана при первом добавлении данных
- Подключение устанавливается лениво — реально соединение происходит при первом запросе

# Выбор коллекции



## Пример

```
products = db["products"]
```

## Особенности

Коллекции тоже создаются автоматически при первом добавлении данных

# Заккрытие соединения



## Пример

```
client.close()
```

## Особенности

MongoClient сам закрывает соединение при завершении программы, но при необходимости его можно закрыть вручную



# Добавление данных



# Добавление данных



В MongoDB все данные хранятся в **документах**, которые представляют собой обычные Python-словари (dict). MongoDB позволяет добавлять документы несколькими способами

# 1. Добавление одного документа — insert\_one

```
product = {
    "name": "Notebook",
    "price": 5.99,
    "stock": 120
}

result = products.insert_one(product)
print("Inserted ID:", result.inserted_id)
```

- Метод `insert_one()` вставляет **один документ**.
- Возвращает объект `InsertOneResult`, из которого можно получить `_id` добавленного документа.

## 2. Добавление нескольких документов — insert\_many

```
items = [
    {"name": "Pen", "price": 1.50, "stock": 300},
    {"name": "Pencil", "price": 0.99, "stock": 500},
    {"name": "Eraser", "price": 0.75, "stock": 200},
]

result = products.insert_many(items)
print("Inserted IDs:", result.inserted_ids)
```

- Метод `insert_many()` принимает список словарей и добавляет их сразу.
- Возвращает `InsertManyResult` с перечнем `_id` всех вставленных документов.

# Особенности



Если поле `_id` не указано вручную, MongoDB создаёт его автоматически.



Вы можете вставить любой словарь, если структура документов в коллекции не фиксирована.



В отличие от SQL, необязательно заранее описывать структуру коллекции.



# ВОПРОСЫ





# Чтение данных

# Чтение данных



Для получения документов из коллекции используются методы `find_one()` и `find()`

# Пример метода find\_one()

```
# Получить один документ  
doc = products.find_one()  
print(doc)
```



# Метод find\_one()

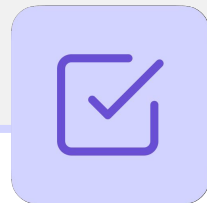
## Особенности:

- Метод find\_one() возвращает **первый попавшийся** документ (или None, если ничего не найдено).
- Можно передавать **условие поиска** — словарь с нужными параметрами

## Пример

```
doc = products.find_one({"price": {"$lt": 5}})
print(doc)
```

# Работа с курсором



Метод `find()` возвращает **объект-курсор**. Это позволяет **не загружать сразу все документы в память**, а перебирать их по одному — как итератор.

# Примеры метода find()

# Получить все документы

```
docs = products.find()
print(docs)
for item in docs:
    print(item)
```

# Получить все документы с условием

```
cursor = products.find({"price": {"$gt": 1}})

for doc in cursor:
    print(doc)
```

# Курсор

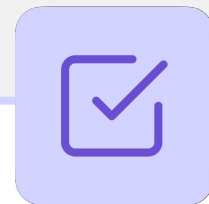
## Особенности:

- Курсор можно использовать в `for`, как обычный итератор.
- После окончания перебора курсор становится **пустым**.
- Можно настроить **лимит, сортировку и пропуск** через методы `.limit()`, `.sort()`, `.skip()`

## Пример

```
for doc in products.find().sort("price", -1).skip(1).limit(2):  
    print(doc)
```

# Проекция полей



По умолчанию `find()` и `find_one()` возвращают **все поля** документа.

Можно указать, **какие поля нужны**, — это называется **проекцией**.

Важно помнить:

- Первый параметр `find()` – словарь с условиями поиска
- Второй параметр `find()` – ограничения проекции
- Если условий поиска нет, то первый параметр нужно передавать пустым словарем

# Примеры

```
# Вернёт только name, price и id (по умолчанию)
for doc in products.find({}, {"name": 1, "price": 1}):
    print(doc)

# Выбрать продукты с ценой больше 100 и исключить _id:
for doc in products.find({"price": {"$gt": 100}}, {"_id": 0}):
    print(doc)

# Оставить только name (все остальные исключаются, включая _id)
for doc in products.find({}, {"_id": 0, "name": 1}):
    print(doc)
```



# Обновление данных

# Обновление данных



Чтобы изменить документ в MongoDB, используются методы `update_one()` и `update_many()`. Они принимают **фильтр** (поиск нужного документа) и **модификатор** (`$set`, `$inc`, и др.).



# Пример: изменить цену одного товара

```
result = products.update_one(
    {"name": "Notebook"},      # фильтр
    {"$set": {"price": 24.99}} # изменение
)

print("Matched:", result.matched_count)
print("Modified:", result.modified_count)
```

- Метод `update_one()` изменяет **только первый подходящий** документ.
- Модификатор `$set` обновляет указанное поле.
- `matched_count` — сколько документов подошли под условие.
- `modified_count` — сколько документов реально изменились.

# Пример: увеличить цену всех товаров

```
result = products.update_many(
    {}, # пустой фильтр – все документы
    {"$inc": {"price": 1}} # увеличить поле price на 1
)

print("Matched:", result.matched_count)
print("Modified:", result.modified_count)
```

- Метод `update_many()` изменяет **все подходящие** документы.
- `$inc` используется для **увеличения числового значения**.
- Возвращаемый объект также содержит `matched_count` и `modified_count`.



# ВОПРОСЫ





# Удаление данных

# Удаление данных



MongoDB позволяет удалять как один документ, так и все, подходящие под условие.

# Пример: удалить один документ

```
result = products.delete_one({"name": "Notebook"})  
print("Deleted:", result.deleted_count)
```

- `delete_one()` удаляет **первый** документ, подходящий под фильтр
- `deleted_count` покажет, сколько документов было удалено (0 или 1)

# Пример: удалить все товары по фильтру

```
result = products.delete_many({"price": {"$lt": 2}})
print("Deleted:", result.deleted_count)
```

- `delete_many()` удаляет **все документы**, удовлетворяющие условию



# Обработка ошибок



# Обработка ошибок



При работе с базой MongoDB через pymongo могут возникать исключения, например, при проблемах с подключением или ошибках в запросах. Чтобы программа не завершалась с ошибкой, нужно использовать блоки `try / except`.

# Пример

```
from pymongo import MongoClient, errors

try:
    client = MongoClient(
        "mongodb://ich_editor:wrong_pass@mongo.itcareerhub.de/?authSource=ich_edit"
    )
    db = client["store"]
    products = db["products"]
    products.insert_one({"name": "Lamp", "price": 15.99})
except errors.ConnectionFailure:
    print("Ошибка подключения к MongoDB")
except errors.OperationFailure:
    print("Ошибка авторизации или запроса")
```

# Основные типы исключений

| Исключение                                    | Описание                                                      |
|-----------------------------------------------|---------------------------------------------------------------|
| <code>pymongo.errors.ConnectionFailure</code> | ошибка подключения к серверу MongoDB                          |
| <code>pymongo.errors.OperationFailure</code>  | ошибка выполнения запроса (например, неправильные права)      |
| <code>pymongo.errors.DuplicateKeyError</code> | попытка вставить документ с уже существующим <code>_id</code> |
| <code>pymongo.errors.PyMongoError</code>      | базовый класс для всех исключений pymongo                     |



# ВОПРОСЫ





# ЗАДАНИЯ





## Выберите верный вариант ответа

Какими способами можно добавить данные в коллекцию MongoDB?

- a. `insert_one()`
- b. `insert_many()`
- c. `add_documents()`
- d. `create_one()`



## Выберите верный вариант ответа

Какими способами можно добавить данные в коллекцию MongoDB?

- a. `insert_one()`
- b. `insert_many()`
- c. `add_documents()`
- d. `create_one()`



## Выберите верный вариант ответа

Какой тип данных возвращается методом `find()`?

- a. Список
- b. Курсор (итератор)
- c. Генератор
- d. Строка JSON
- e. Словарь





## Выберите верный вариант ответа

Какой тип данных возвращается методом `find()`?

- a. Список
- b. Курсор (итератор)**
- c. Генератор
- d. Строка JSON
- e. Словарь



# Модули



## Модуль

Это любой `.py` файл, содержащий переменные, функции, классы и другие конструкции

# Преимущества модулей



Упрощают структуру программы и позволяют разделять код по смыслу



Повышают читаемость и удобство сопровождения кода



Облегчают повторное использование и помогают избегать дублирования



Позволяют использовать готовые решения из стандартной библиотеки Python



Упрощают тестирование

# Виды модулей

## Встроенные модули

Входят в стандартную библиотеку Python (например, `math`, `random`, `datetime`)

## Сторонние модули

Устанавливаются через менеджер пакетов `pip`

## Пользовательские модули

Это любые `.py`-файлы, написанные самостоятельно

# Работа с собственным модулем

Чтобы импортировать свой модуль, он должен находиться в той же папке или в доступном пути.

Создадим файл `math_utils.py` со следующим содержимым:

```
print("Inside math_utils.py")

def average(numbers):
    return sum(numbers) / len(numbers)

def maximum(numbers):
    return max(numbers)
```

# Работа с собственным модулем

Теперь в другом файле (в той же папке), например `main.py`, можно использовать этот модуль.

```
import math_utils
from math_utils import maximum

values = [10, 20, 30]

print(math_utils.average(values))
print(maximum(values))
```

# Прямой запуск модуля

Иногда модуль может использоваться и как **библиотека**, и как самостоятельный **скрипт**, запускаемый напрямую. Чтобы различать эти случаи, в Python используют конструкцию:

```
if __name__ == "__main__":  
    # Код, который будет выполняться только при прямом запуске
```



# Прямой запуск модуля



Когда модуль запускается напрямую, переменная `__name__` получает значение `"__main__"`.

Если модуль **импортируется** в другой файл — `__name__` будет равно имени модуля.

# Пример

Файл `math_utils.py`:

```
# Это сообщение будет выведено при прямом запуске или при импорте
print("Inside math_utils.py")
```

```
def average(numbers):
    return sum(numbers) / len(numbers)
```

```
def maximum(numbers):
    return max(numbers)
```

```
# Это сообщение будет выведено только при прямом запуске
if __name__ == "__main__":
    print("Inside main file!")
```

Только если запустить `math_utils.py` напрямую — будет выведено "Inside main file!".

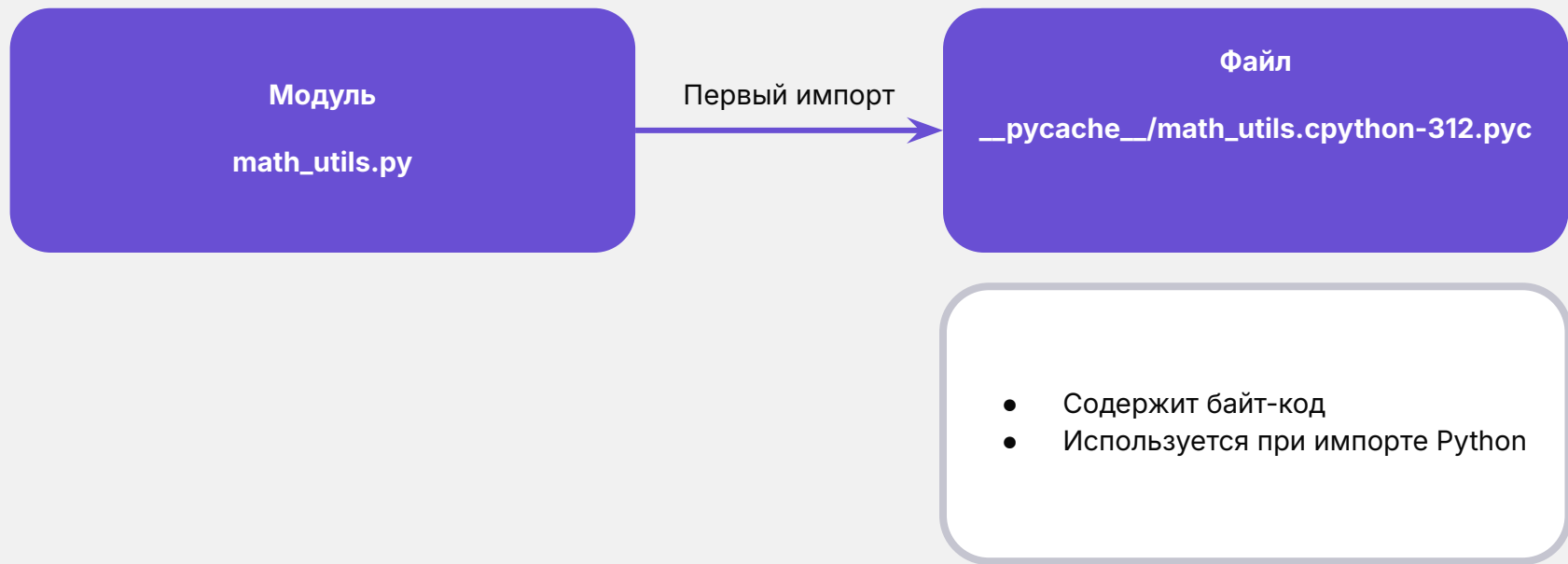
# Компиляция модулей



Когда Python выполняет импорт модуля, он автоматически компилирует его в байт-код — промежуточный формат, который быстрее загружается при следующем запуске.

Скомпилированный файл сохраняется с расширением `.pyc` и помещается в папку `__pycache__`.

# Пример



# .рус файл пересоздается



При первом импорте модуля



Если содержимое .ру-файла изменилось



Если обновилась версия Python



# ВОПРОСЫ





# ЗАДАНИЯ





## Выберите верный вариант ответа

**Что такое модуль в Python?**

- a. Это встроенный тип данных, предназначенный для хранения больших чисел.
- b. Это отдельный файл с кодом на Python, содержащий переменные, функции или классы, которые можно использовать в других программах.
- c. Это специальная область памяти, где хранятся глобальные переменные.
- d. Это пользовательский файл, где хранятся параметры конфигурации и служебные функции.





## Выберите верный вариант ответа

**Что такое модуль в Python?**

- a. Это встроенный тип данных, предназначенный для хранения больших чисел.
- b. Это отдельный файл с кодом на Python, содержащий переменные, функции или классы, которые можно использовать в других программах.**
- c. Это специальная область памяти, где хранятся глобальные переменные.
- d. Это пользовательский файл, где хранятся параметры конфигурации и служебные функции.



## Выберите верный вариант ответа

Что делает `__pycache__/math_utils.cpython-312.pyc`?

- a. Хранит текст модуля в сжатом виде
- b. Является скомпилированной версией .py файла
- c. Позволяет запускать программу без Python
- d. Содержит переменные окружения



## Выберите верный вариант ответа

Что делает `__pycache__/math_utils.cpython-312.pyc`?

- a. Хранит текст модуля в сжатом виде
- b. Является скомпилированной версией .py файла
- c. Позволяет запускать программу без Python
- d. Содержит переменные окружения



# ПРАКТИЧЕСКАЯ РАБОТА



# 1. Поиск заказов с маленькой суммой

Прочитайте все документы из коллекции orders, у которых сумма (amount) **меньше 10**. Выведите каждый найденный заказ построчно.

**Пример вывода:**

```
{'_id': ObjectId('...'), 'id': 3, 'customer': 'Olga', 'product': 'Kiwi', 'amount': 9.6, 'city': 'Berlin'}  
{'_id': ObjectId('...'), 'id': 5, 'customer': 'Olga', 'product': 'Banana', 'amount': 8, 'city': 'Madrid'}
```

## 2. Сохранение результатов в другую коллекцию

Сохраните все найденные заказы в новую коллекцию `orders_lesson_38`. После записи выведите, сколько документов было добавлено.

**Пример вывода:**

```
6 documents inserted into 'orders_lesson_38'.
```



# ДОМАШНЕЕ ЗАДАНИЕ



# Домашнее задание

## 1. Добавление товаров

Создайте программу, которая подключается к MongoDB и:

- выбирает базу `ich_edit` и коллекцию `products_<your_group>_<your_full_name>`
- очищает коллекцию перед началом
- добавляет 3 товара с полями: `name`, `price`, `stock`
- выводит сообщение о количестве добавленных товаров

**Пример вывода:**

```
3 products inserted.
```



# Домашнее задание

## 2. Увеличение цен

Продолжите предыдущую задачу. Теперь программа должна:

- увеличить цену **всех товаров** на 20%
- вывести количество обновлённых записей
- затем вывести список всех товаров с новыми ценами

### Пример вывода:

Prices updated for 3 products.

Updated products:

- Pen - \$1.80
- Notebook - \$4.79
- Backpack - \$30.00

## Заключение

